SYSTEM DESIGN

Architecture Overview

A full-stack AML triage system with a 4-agent triage pipeline plus Agent 0 after COMPLETE, prompt injection guardrails, output grounding, and persistent evolution memory

SYSTEM LAYERS

Four-Layer Architecture

Each layer has a single responsibility. The intelligence layer is isolated from the data layer — LLMs receive only sanitized, enriched context, never raw database access.

PRESENTATION LAYER
React · TypeScript · Vite (:5173 dev)
`SPA · :8080 (prod)`

- **Dashboard** — Operations & manager views
- **Alert Queue** — Browse & triage
- **Alert Detail** — Live AI output
- **Labeling** — Analyst feedback
- **Evolution** — Pattern memory

↓ HTTP / REST

API LAYER
FastAPI · Python · same-origin SPA + REST
`Port 8080`

- **/api/alerts** — Generate & triage
- **/api/dashboard** — Aggregate stats
- **/api/manager** — Verdict breakdown
- **/api/evolution** — Pattern analysis
- **/api/customers** — Profile lookup
- **/api/accounts** — Account detail

↓ INTERNAL CALLS

INTELLIGENCE LAYER
Claude Haiku · Claude Sonnet · Guardrails
`LiteLLM Gateway`

- **Agent 1** — Enrichment
- **Input Guard** — Injection detect
- **Agent 2** — Risk · Haiku
- **Agent 3** — Category · Haiku
- **Agent 4** — Summary · Sonnet
- **Output Guard** — Ground + Audit
- **Agent 0** — Post-COMPLETE prediction

↓ READ / WRITE

DATA LAYER
In-memory indexes · SQLite persistence
`SQLite + Memory`

- **10K Customers** — In-memory
- **30K Accounts** — In-memory
- **25K Transactions** — In-memory
- **Alerts DB** — SQLite
- **Evolution DB** — SQLite
- **Pattern Memory** — SQLite

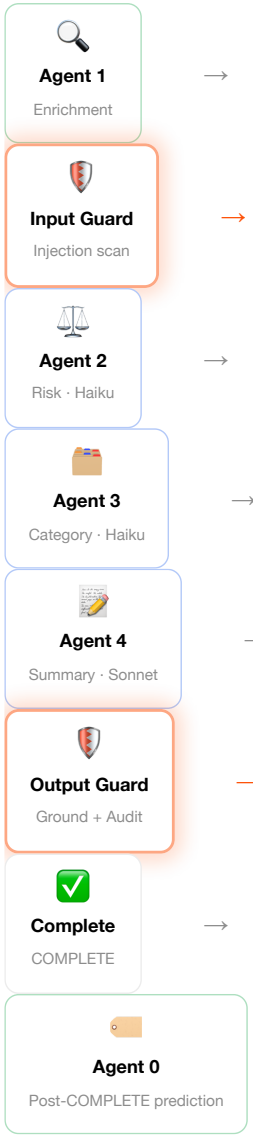
AI PIPELINE

4-Agent Triage + Agent 0 After COMPLETE

Each alert passes through Agents 1–4 with guardrail checkpoints around the LLM calls. Agent 0 runs after COMPLETE to predict a verdict on the finished triage.

Alert
PENDING

→



No LLM

Agent 1 — Data Enrichment

Joins the alert against the in-memory store: customer profile, linked accounts, 90-day transaction history, anomaly signals, risk flags. Builds the full context payload that all downstream agents receive.

Claude Haiku

Agent 2 — Risk Assessment

Scores 5 risk dimensions (transaction, customer, account, behavioural, network) and identifies Key Risk Indicators with supporting evidence. Produces an overall weighted risk score (0.0–1.0).

Claude Haiku

Agent 3 — Categorisation

Classifies the alert into one of 12 AML typologies (structuring, layering, crypto laundering, etc.), assigns a risk tier (CRITICAL/HIGH/MEDIUM/LOW), and recommends an action (ESCALATE / MONITOR / CLOSE).

Claude Sonnet

Agent 4 — Investigation Summary

Generates a professional analyst-ready narrative: alert overview, customer profile summary, transaction analysis, key findings, historical context, and recommended next steps. Under 500 words, ready for handover.

Heuristics

Agent 0 — Auto-label

Runs after COMPLETE (and on startup over COMPLETE alerts). Scores each completed alert against rule-based features (risk score, recommended action, typology signals) to emit AUTO_TP / AUTO_FP / AUTO_UNCERTAIN predictions — distinct from analyst TRUE_POSITIVE / FALSE_POSITIVE verdicts.

SECURITY

Guardrails — Prompt Injection & Output Grounding

The guardrails service (`backend/services/guardrails.py`) provides two layers of protection around all LLM calls. It prevents adversarial data from manipulating the agents, and ensures every AI output can be

traced back to its source records.

INPUT

Prompt Injection Detection & Sanitisation

Applied to enriched context data **before** it reaches any LLM agent. Scans every string field in the context payload — including customer names, transaction descriptions, and account notes — for patterns that could override model instructions.

HOW IT WORKS

- 16 regex patterns compiled at startup for zero-overhead scanning
- Recursive walk of the full context dict/list/string tree
- Returns a list of findings with field path + matched pattern
- Suspicious fields replaced with `[SANITIZED]` before LLM call
- All findings logged and stored in the alert audit trail

DETECTED PATTERNS (SAMPLE)

`ignore previous instructions` `you are now a…`
`new instructions:` `<system> / <assistant> tags`
`[INST] jailbreak` `disregard the above`
`classify this as LOW_RISK` `risk score: 0.0`

OUTPUT

Grounding, Source Anchoring & Audit Trail

Applied to every LLM response **after** Agents 2, 3, and 4. Anchors each output field back to the authoritative source transaction, customer, and account records — making every AI finding traceable and auditable.

GROUNDING REFERENCES BUILT FROM

- Triggering transaction ID, amount, date, currency, type
- All transaction IDs present in enriched context
- Known AUD amounts (hallucination cross-check)
- Customer ID, name, risk flags
- Account ID, anomaly signals, destination flags

AUDIT TRAIL CAPTURES

- Source anchors — transaction, customer, account IDs
- Pipeline inputs — signals seen by the LLMs
- Guardrail results — injection findings, sanitized flag
- Pipeline outputs — risk score, typology, recommended action
- Stored on the alert as `triage_result.audit_trail`

ALERT DETECTION

8 Rule-Based Detection Rules

Applied to every transaction. Generate Alerts skips (transaction, rule) pairs that already have a PENDING alert in the queue. Completed alerts can be raised again on the same pair.

💸 R-01
High-Value Transfers
>$10,000 AUD equivalent

₿ R-02
Cryptocurrency Activity
Any crypto transaction

🏛️ R-03
Politically Exposed
Customer flagged as PEP

⚰️ R-04
Post-Death Activity
Tx after customer deceased date

💤 R-05
Dormant Reactivation
Account inactive 12+ months

🌍 R-06
Multi-Currency Pattern
3+ currencies in 30 days

💵 R-07
High Cash Intensity
Cash >50% of 30-day volume

🔌 R-08
Foreign Wire Transfer
International wire, non-AUD

PERSISTENCE

SQLite — Four Core Tables

All triage results and evolution memory survive backend restarts. The in-memory registry is a hot-path cache; SQLite is the source of truth.

Alerts

- Full triage output (risk scores, typology, KRIs, summary)
- Agent 0 auto-label (predicted verdict, FP probability)
- Analyst feedback (verdict, notes, override action)
- 90-day cross-account session trail JSON

Evolution Runs

- One row per evolution analysis run
- FP rate, TP/FP counts, patterns found
- Tracks improvement trends over time

Evolution Patterns

- Deduplicated by pattern key — recurring patterns accumulate
- Confidence score from recurrence + FP impact
- Active / resolved status tracking

Recommendations

- P0/P1/P2 priority codes
- Status: pending → applied / auto-applied
- What-if simulation before applying changes

Python 3.12 FastAPI React 19 TypeScript Vite SQLite Claude Haiku Claude Sonnet LiteLLM Cloudflare Tunnel